REPORT

All experiments were carried out on **the university** 's Linux machines. Indicatively, one machine has:

Number of cores : 4

Processor : Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz

OS : NAME="Ubuntu" VERSION="20.04.6 LTS (Focal Fossa)"

Compiler Version : 9.4.0

Also, following the *lscpu command | grep Flags,* I saw that on Linux machines sse, sse2, sse3, ssse3, sse4.1, sse4.2, avx, avx2 and others are supported.

Exercise 4.1

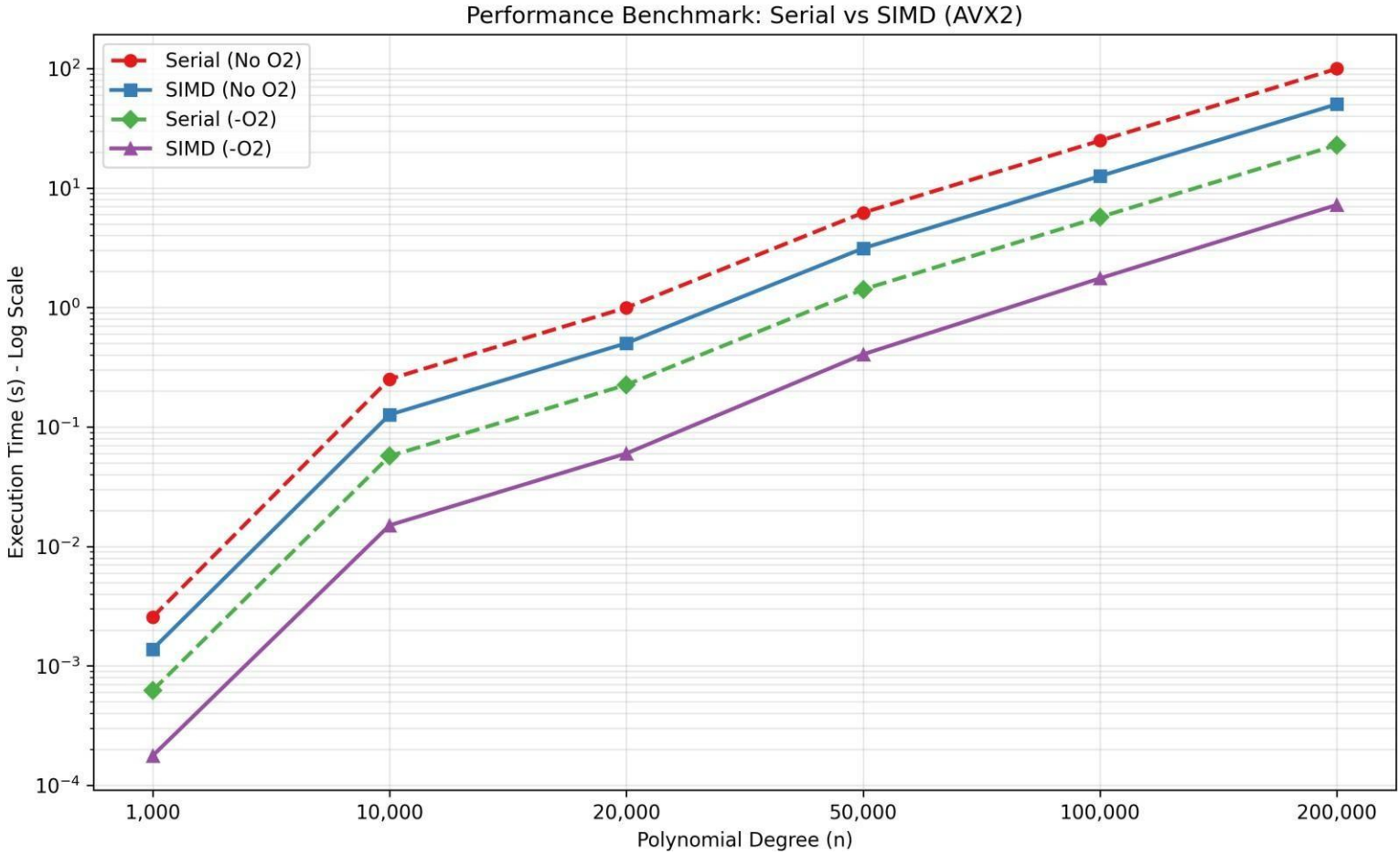
Initially, in relation to the previous work, I did not use the formula **__int128_t** for results because they are data types offered by the compiler and there are no **avx2** registers that can directly add to 128 bit numbers. I had used them for 100% certainty that there will be no overflow but now for simplicity I have replaced them with int64_t for simplicity and since you told me that there is no problem. I mention this because there is an reduction in the time of the serial algorithm for not using **nt128_t** (for 105 out of 34 seconds in 25) but as long as I do not compare times with the previous tasks, there is no problem since in this particular one everything has been implemented with the same formula to have correct and logical comparisons.

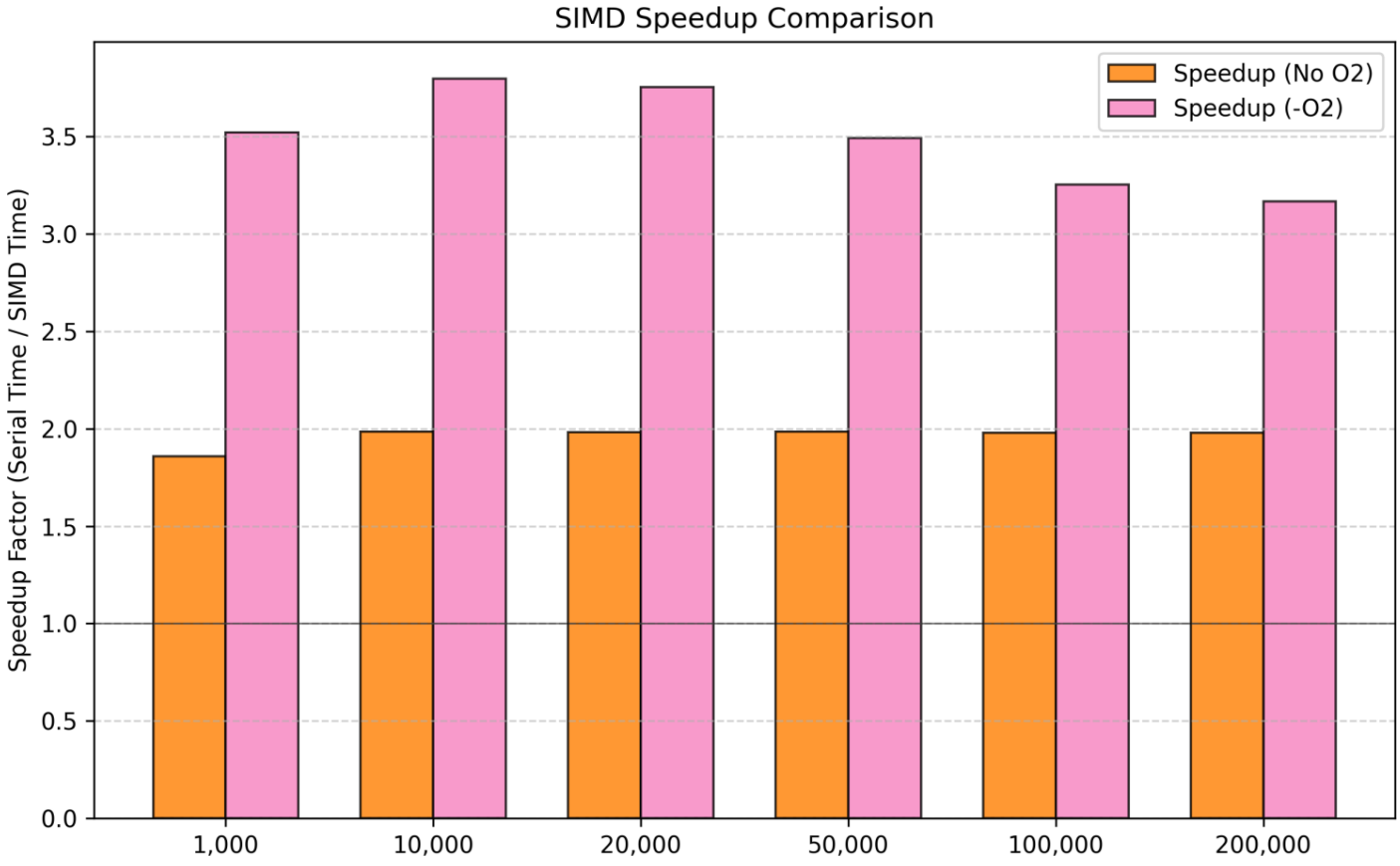
In the beginning I had the program run only the serial algorithm without comparing results, and I compared the execution with **flags** in compilers for vectorise and Optimization (-**O2**). Indicatively, for n = 100000, the time without flag
In compiler it was **25** seconds while with flag it was **6**. Then I opened the object file to see if there were indeed simd commands and indeed there were, so the improvement came _from there as well.

Something that I improved and reduced our time a lot is that in polynomial **2**, because the values that need to be loaded into the vectorized register are with descending indices {k-i, k-i-1... k-i-7}, so I couldn't
use the **_mm256_loadu_si256** approach to load contiguous memory space (8 int32 values in this case) but
I called **M256i B = _mm256_setr_epi32** which apparently was much slower since it copied the values one by one. So in the end, I used a
poly2_reversed table that I saved the table inverted, so that I could do a load as with Poly_1. This for n=105 reduced the time of parallel execution **2G->18 seconds by 10 seconds!**Then, something that reduced the time even more (**18->12**) was the following:

At first, because the 32-bit multiplication command in avx2 can't be
save results for all numbers because it would like 8*64 (and it doesn't have 512 bit registers) so it only keeps in the even lanes result, I manually cast in numbers 64 bits and kept in registers high and low bits, and I did the corresponding multiplications. Now I do the
multiplication, then I slide and multiply again because the odd lanes (numbers 1,3,5,7) became even. This saves us copying to extra registrars.

Something I also noticed is that when I put both -O2 flag and flag for Simd extensions, the time of the serial is reduced a lot (it uses simd commands even though I don't tell it explicitly) and the time of the parallel algorithm with intrinsics avx2 -O2 is by far the best. For this I ran the code both with Optimization of the compiler and without:


Performance Benchmark: Serial vs SIMD (AVX2)
Serial (No O2)
SIMD (No O2)
Serial (-O2)
SIMD (-O2)
Execution Time (s) - Log Scale
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
10^{-3}
10^{-4}
1,000
10,000
20,000
50,000
100,000
200,000
Polynomial Degree (n)


SIMD Speedup Comparison
Speedup (No O2)
Speedup (-O2)
Speedup Factor (Serial Time / SIMD Time)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
1,000
10,000
20,000
50,000
100,000
200,000

It is obvious that Simd with optimization works much better and in fact its speedup compared to serial with Optimization is quite large. However, in the difference between serial and parallel without optimization, there is a noticeable
Difference

Exercise 4.2

The final nvidia exercise was done by an nvidia account collaboratively and involves accelerating a simulator of Maxwell's equations in a two-dimensional grid. The simulator is initially implemented for CPUs and is slow, especially for large grids. The goal is to port the running of the simulator to a GPU so that a significant improvement in performance can be achieved. The assessment consisted of 3 parts:

The initial implementation uses CPU data structures and transfers
data between the CPU and the GPU. I had to replace the std::vector of floats with Thrust::d evice_vector, since the GPU needs them for computation, and they were not declared in **Unified Memory** so that there would be no unexpected implicit copies as mentioned in chapter **01.06**. I also use Thrust::transform instead of std::transform and thrust::for_each respectively. In step 2
Counting and zip iterators were used so that vector buffers and cell_ids were no longer needed and unnecessary memory allocations were not made in memory. At update_hy and update_hx the two transform calls were replaced by a single for_each call, since there is no longer the need for an intermediate buffer. Finally, in step 3, the analysis of the simulation result is reduced in order to make the program more efficient,
making tiles by taking the average from a specific piece of the grid.
CUB's BlockReduce is used.